

Task 1: OpenCV – The Enigma of Professor SS Lachure's PC

Project Submission Report

Submitted By: Darshan Mahale

Track: Track B – General Tasks

Task: OpenCV Image Processing Maze Challenge

Final Password: 631772213

Submission Files

ZIP Archive

[Click here](#)

Contents of Submission

- MazeMaker.exe
- mazes/ (100 generated maze images)
- answers/ (100 generated solution images)
- solution_code.py
- password.txt

Project Summary

The objective of this task was to process 100 noisy maze images, identify the shortest path from the top-left cell to the bottom-right cell, generate visual solution images, handle impossible mazes, and compute a final password using modulo arithmetic.

Rather than directly writing a final solver, the project was approached through a series of experiments and verification steps. These included image analysis, color inspection, noise removal, grid extraction, logical maze generation, shortest-path computation using Breadth First Search (BFS), solution visualization, automation for processing all 100 mazes, and final password generation.

The complete development process, experiments performed, challenges encountered, debugging steps, and final solution architecture are documented in the following sections.

Introduction

The objective of this task was to develop an automated system capable of solving 100 maze images provided in the dataset. Each maze image contained visual noise and represented a logical 40×40 maze grid embedded within an 800×800 pixel image. The final goal was not only to solve individual mazes but also to compute a password generated from the shortest path lengths of all solvable mazes.

Rather than directly implementing a maze solver, the project was approached as a sequence of experimental stages. The first step involved understanding the structure of the images, including their dimensions, color composition, and noise characteristics. Once the image properties were understood, the next challenge was to clean the images and convert them into a logical maze representation that could be processed algorithmically.

After constructing a reliable maze representation, Breadth First Search (BFS) was implemented to compute the shortest path between the start cell and the goal cell. The resulting path was then visualized on the original maze image. Finally, the entire pipeline was automated to process all 100 mazes, handle impossible cases, generate solution images, and compute the final password using modulo arithmetic.

The complete development process is documented in the following sections, including the experiments performed, design decisions taken, bugs encountered, and the reasoning behind the final solution.

Task 1 – OpenCV Maze Solver Development Workflow



IMAGE ANALYSIS

PHASE 1 – DATASET ANALYSIS & PROBLEM UNDERSTANDING

- Load sample maze image
- Verify image dimensions (800×800)
- Inspect pixel values
- Analyze color distribution
- Discover 103 unique colors caused by noise
- Identify wall color and path color



IMAGE CLEANING & GRID EXTRACTION

PHASE 2 – IMAGE CLEANING, GRID EXTRACTION & MAZE REPRESENTATION

- Remove grayscale noise using thresholding
- Generate cleaned maze image
- Verify only red and white remain
- Count wall and path pixels
- Measure wall width and cell spacing
- Determine logical cell size = 20×20 pixels
- Convert image into 40×40 logical maze matrix



BFS SEARCH

PHASE 3 – WALL DISCOVERY, MAZE VALIDATION & BFS PATHFINDING

- Validate start cell and goal cell
- Mark walls as 1 and paths as 0
- Build graph implicitly from maze matrix
- Apply Breadth First Search (BFS)
- Track visited nodes and parent nodes
- Compute shortest path length
- Detect impossible mazes



SOLUTION VISUALIZATION

PHASE 4 – SOLUTION VISUALIZATION

- Reconstruct shortest path
- Convert logical path back to image coordinates
- Paint path in green
- Generate solved maze image
- Visually verify correctness



AUTOMATION & OUTPUT

PHASE 5 – FULL AUTOMATION & PASSWORD GENERATION

- Process all 100 maze images automatically
- Handle solvable and impossible mazes
- Save outputs in answers folder
- Multiply path lengths of solvable mazes
- Apply modulo 1,000,000,007
- Generate password.txt
- Produce final submission package

Phase 1: Dataset Analysis & Investigation

Before attempting to solve the maze, I first analyzed the dataset to understand how the maze was represented within the image and to identify any challenges that would affect the solution.

The first step was to load a sample maze image and inspect its basic properties. This included verifying that the image was being read correctly, checking its dimensions, and examining pixel values at different locations within the image. Through this analysis, I confirmed that each maze image had a resolution of 800×800 pixels.

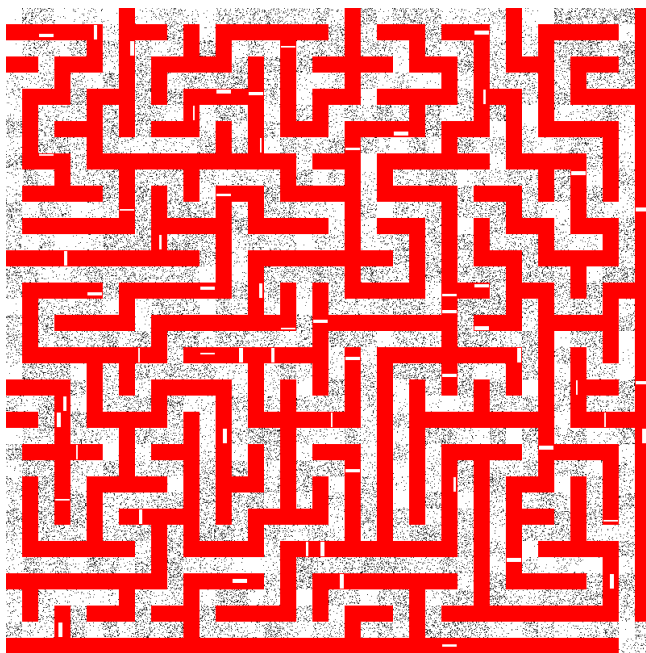


Figure 1. Original maze image provided in the dataset before any preprocessing.

```
Solver > archive.py > ...
1  import cv2
2
3  image = cv2.imread("X task 1 Windows/mazes/maze_01.png")
4
5  print("Image loaded successfully:", image is not None)
6
7  print("Shape:", image.shape)
8
9  print("Top-left pixel value:", image[0, 0])
10
11 print("Center pixel value:", image[400, 400])
12
13 print("Pixel at 100,100:", image[100,100])
14 print("Pixel at 200,200:", image[200,200])
15 print("Pixel at 300,300:", image[300,300])
16 print("Pixel at 50,50:", image[50,50])
17 print("Pixel at 51,51:", image[51,51])

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
python
sk 1 Open CV\Solver\archive.py"
Image loaded successfully: True
Shape: (800, 800, 3)
Top-left pixel value: [255 255 255]
Center pixel value: [255 255 255]
Pixel at 100,100: [ 0  0 255]
Pixel at 200,200: [255 255 255]
Pixel at 300,300: [ 0  0 255]
Pixel at 50,50: [22 22 22]
Pixel at 51,51: [255 255 255]
Pixel at 52,52: [255 255 255]
Pixel at 53,53: [255 255 255]
PS D:\Project X\Track B General tasks\Task 1 Open CV>
```

Figure 2. Output of the initial dataset inspection script showing successful image loading, image dimensions, and sample pixel values extracted from different regions of the image.

[Click Here \(archive.py\)](#)

After understanding the image structure, I performed a color analysis on the original maze image. At first glance, the maze appeared to contain only red walls and white paths. However, a detailed inspection revealed that the image contained many additional grayscale color values.

The presence of these extra colors indicated that the image contained visual noise. This observation was important because it meant that the maze could not be directly converted into a logical representation. A preprocessing step would be required before maze extraction could begin.

```
3
4 image = cv2.imread("original_maze.png")
5
6 pixels = image.reshape(-1, 3)
7
8 unique_colors = np.unique(pixels, axis=0)
9
10 print("Number of unique colors:", len(unique_
11
12 print(unique_colors)
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
sk 1 Open CV\Solver\read_maze.py"
Number of unique colors: 103
[[ 0  0  0]
 [ 0  0 255]
 [ 1  1  1]
 [ 2  2  2]
 [ 3  3  3]
 [ 4  4  4]
 [ 5  5  5]
 [ 6  6  6]
 [ 7  7  7]
 [ 8  8  8]
 [ 9  9  9]
```

Figure 3. Unique color analysis of the original maze image showing the presence of more than one hundred unique colors.

[Click Here \(read_maze.py\)](#)

Key Findings

- Maze image resolution is 800×800 pixels.
- The maze represents a logical 40×40 grid.
- Maze walls are represented using red pixels.
- Walkable paths are represented using white pixels.
- Significant grayscale noise is present throughout the image.
- Image cleaning would be required before extracting the maze structure.

These observations provided the foundation for the next phase, where the image was cleaned and transformed into a machine-readable maze representation.

Phase 2: Image Cleaning, Grid Extraction & Maze Representation

After analyzing the dataset in Phase 1, the next objective was to remove the image noise and convert the maze into a logical representation that could be processed algorithmically.

The color analysis performed earlier revealed that the image contained 103 unique colors. However, only two of these colors represented actual maze information: red pixels $(0, 0, 255)$ corresponding to walls and white pixels $(255, 255, 255)$ corresponding to walkable paths. The remaining colors were grayscale values ranging approximately from $(0, 0, 0)$ to $(100, 100, 100)$, indicating the presence of visual noise.

Based on this observation, a threshold-based cleaning approach was developed. Any pixel whose RGB values were less than or equal to 100 was classified as noise and converted to white. This preserved the maze structure while eliminating the unwanted grayscale artifacts.

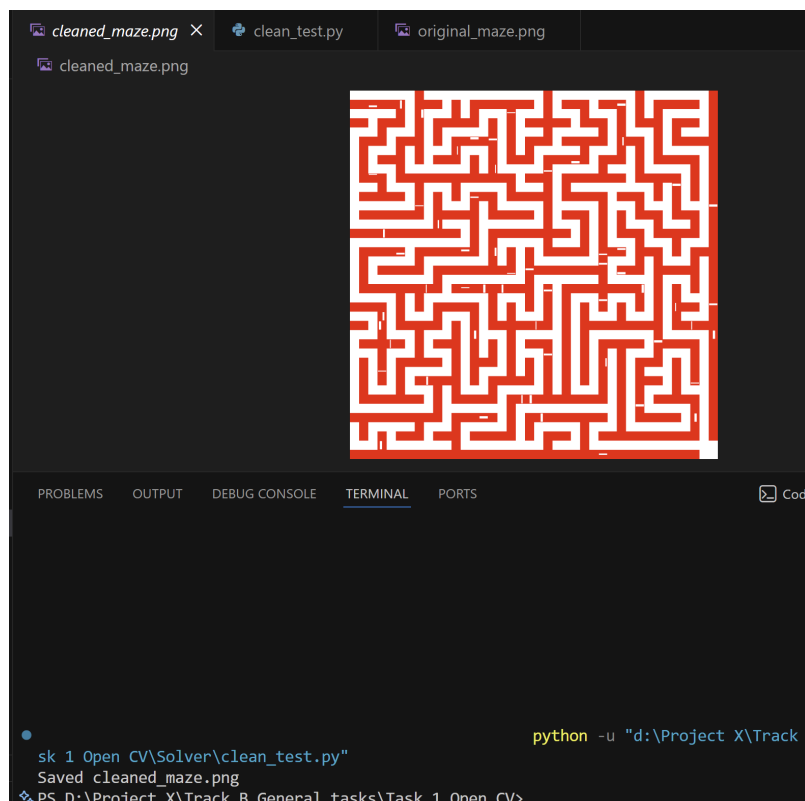
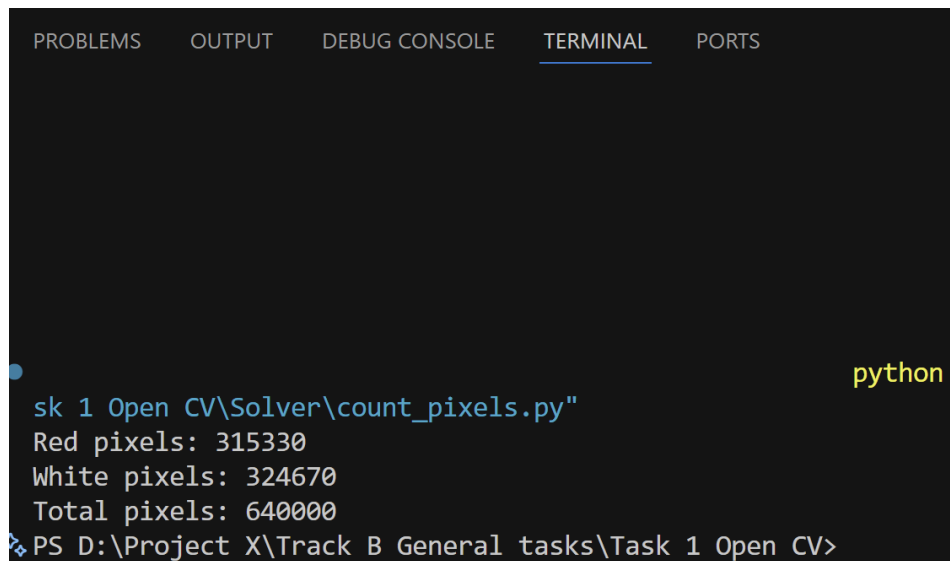


Figure 4. Comparison of the original maze image and the cleaned maze image after the noise removal process.

[Click Here \(clean_test.py\)](#)

After cleaning the image, it was necessary to verify that the preprocessing stage had worked correctly. A pixel analysis script was used to count the number of red and white pixels

present in the cleaned image. The results confirmed that the image now consisted entirely of maze walls and walkable paths, with the grayscale noise successfully removed.



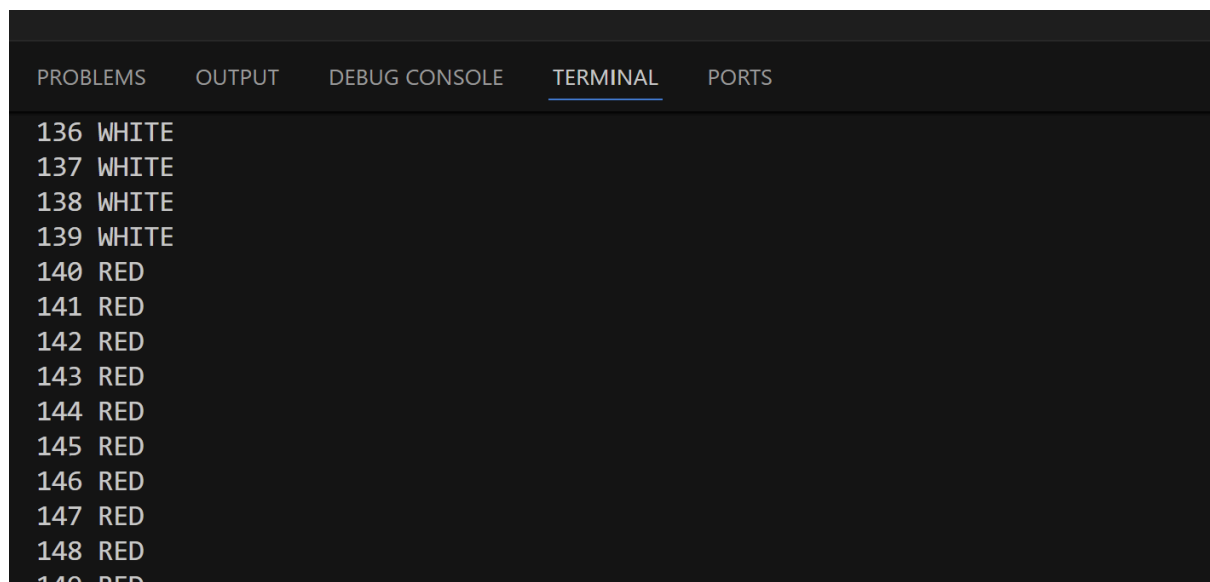
```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

python
sk 1 Open CV\Solver\count_pixels.py"
Red pixels: 315330
White pixels: 324670
Total pixels: 640000
PS D:\Project X\Track B General tasks\Task 1 Open CV>
```

Figure 5. Pixel-count verification performed on the cleaned image to confirm successful noise removal.

[Click Here \(count_pixels.py\)](#)

With the image cleaned, the next challenge was to determine how the logical maze grid was embedded within the 800×800 image. To investigate this, a row-wise analysis of pixel values was performed. By observing transitions between red wall regions and white path regions, it became possible to estimate the spacing and structure of the maze.



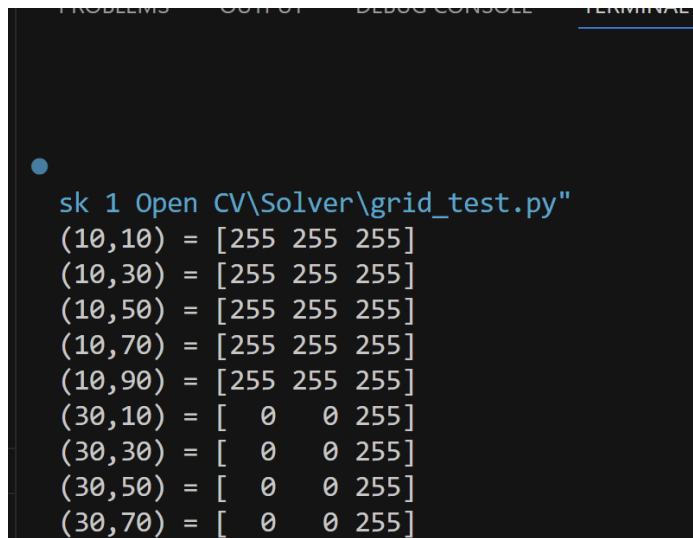
```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

136 WHITE
137 WHITE
138 WHITE
139 WHITE
140 RED
141 RED
142 RED
143 RED
144 RED
145 RED
146 RED
147 RED
148 RED
149 RED
```

Figure 6. Row-wise pixel inspection used to study wall widths and maze structure.

[Click Here \(measure_width.py\)](#)

To further validate the grid structure, pixel values were sampled at regularly spaced coordinates throughout the image. The results showed a repeating pattern every 20 pixels, suggesting that each logical maze cell occupied a 20×20 pixel region.

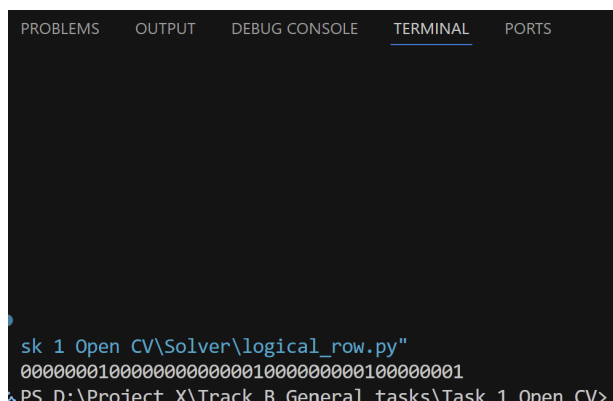


```
sk 1 Open CV\Solver\grid_test.py"
(10,10) = [255 255 255]
(10,30) = [255 255 255]
(10,50) = [255 255 255]
(10,70) = [255 255 255]
(10,90) = [255 255 255]
(30,10) = [  0  0 255]
(30,30) = [  0  0 255]
(30,50) = [  0  0 255]
(30,70) = [  0  0 255]
```

Figure 7. Grid validation experiment used to confirm the spacing of logical maze cells.

[Click Here \(grid_test.py\)](#)

Before extracting the entire maze, a smaller experiment was performed by converting a single logical row into a sequence of wall and path values. This confirmed that image pixels could be successfully translated into a machine-readable representation.

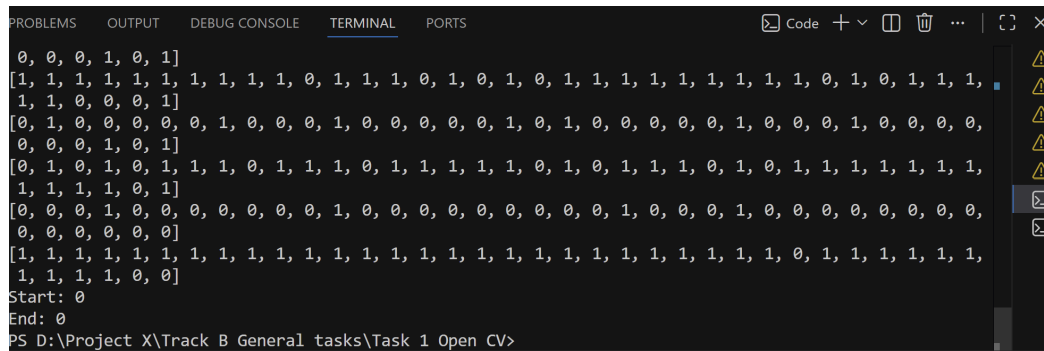


```
sk 1 Open CV\Solver\logical_row.py"
00000000100000000000000010000000001000000001
PS D:\Project X\Track B General tasks\Task 1 Open CV>
```

Figure 8. Example of a logical maze row extracted from the cleaned image.

[Click Here \(logical_row.py\)](#)

After validating the extraction process on a single row, the same approach was applied to the complete image. Each logical cell was classified and converted into a 40×40 matrix representation, where walls and paths could be represented numerically and processed by pathfinding algorithms.



```
0, 0, 0, 1, 0, 1]
[1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 0, 1, 1, 1, 0, 1, 0, 1, 0, 1, 1, 1, 1, 1, 1, 1, 0, 1, 0, 1, 1, 1,
1, 1, 0, 0, 0, 1]
[0, 1, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 1, 0, 1, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0,
0, 0, 0, 1, 0, 1]
[0, 1, 0, 1, 0, 1, 1, 1, 0, 1, 1, 1, 0, 1, 1, 1, 1, 0, 1, 0, 1, 1, 1, 0, 1, 1, 1, 1, 1, 1, 1, 1,
1, 1, 1, 1, 0, 1]
[0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0, 0]
[1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 0, 1, 1, 1, 1, 1, 1,
1, 1, 1, 1, 0, 0]
Start: 0
End: 0
PS D:\Project X\Track B General tasks\Task 1 Open CV>
```

Figure 9. Generated 40 × 40 logical maze representation showing wall and path classifications.

[Click Here \(extract_maze.py\)](#)

Key Findings

- The original image contained 103 unique colors.
- Only two colors represented actual maze information: red walls and white paths.
- Grayscale values were identified as image noise and removed using thresholding.
- The cleaned image contained only wall and path information.
- The maze was confirmed to be a 40 × 40 logical grid.
- Each logical cell occupied a 20 × 20 pixel region.
- A complete machine-readable maze matrix was successfully generated.

At the end of this phase, the maze had been transformed from a noisy image into a structured logical representation. This representation would be used in the next phase to validate the maze structure, identify extraction issues, and compute the shortest path using Breadth First Search (BFS).

Phase 3 – Path Discovery, Validation and Bug Fixing

After converting the cleaned maze image into a logical 40×40 grid, the next objective was to determine the shortest path from the start cell (top-left) to the goal cell (bottom-right). Since each cell could either be a wall or a walkable path and movement was restricted to the four cardinal directions (up, down, left and right), the problem could be modeled as a graph traversal problem.

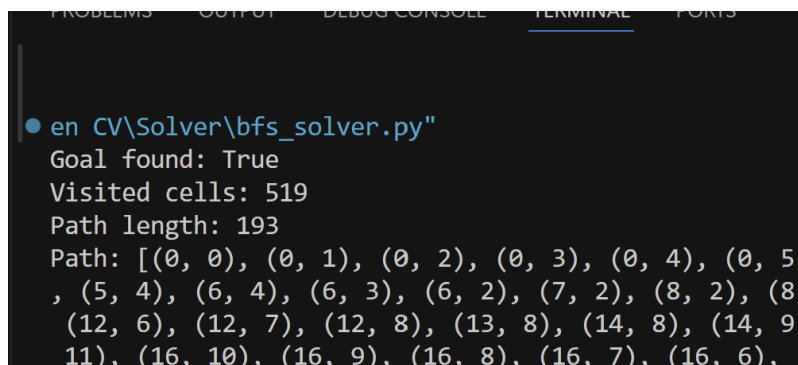
I selected **Breadth First Search (BFS)** because BFS guarantees the shortest path in an unweighted grid. This makes it one of the most commonly used algorithms for maze-solving applications.

The algorithm first verified that both the start cell and goal cell were walkable. BFS was then executed beginning from the start cell. At each step, neighbouring cells were explored level by level while maintaining:

- A queue for traversal
- A set of visited cells
- A parent mapping used for path reconstruction

Whenever a new valid neighbour was discovered, it was added to the queue, marked as visited and linked to its parent cell. Once the goal cell was reached, the shortest path could be reconstructed by backtracking through the parent mapping from the goal node to the start node.

Code Reference: [Click Here \(bfs_solver.py\)](#)



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
● en CV\Solver\bfs_solver.py"
Goal found: True
Visited cells: 519
Path length: 193
Path: [(0, 0), (0, 1), (0, 2), (0, 3), (0, 4), (0, 5),
(5, 4), (6, 4), (6, 3), (6, 2), (7, 2), (8, 2), (8,
(12, 6), (12, 7), (12, 8), (13, 8), (14, 8), (14, 9),
11), (16, 10), (16, 9), (16, 8), (16, 7), (16, 6),
```

Figure 10: BFS successfully traversing the logical maze representation and computing the shortest path length.

Bug Discovery During Validation

Although BFS successfully found valid paths within the logical maze representation, visual validation revealed an unexpected issue. Some generated paths appeared to travel through wall regions when mapped back onto the maze image.

Since BFS only traverses cells that are marked as walkable, this observation suggested that the problem was not in the pathfinding algorithm itself. Instead, the issue originated from the maze extraction process used to construct the logical grid.

To investigate the problem, I examined several wall boundaries at a higher zoom level and compared the extracted logical cells against the original image.



Figure 11: Zoomed wall boundary showing thin white artifacts inside a predominantly red wall region.

The initial maze extraction method classified each logical cell using only the center pixel of the corresponding 20×20 block. This approach worked correctly for most cells but failed near wall boundaries.

Several wall regions contained thin white strips caused by image artifacts and rendering noise. If the center pixel happened to fall inside one of these strips, the entire cell would be incorrectly classified as a walkable path even though most of the cell represented a wall.

As a result, the logical maze occasionally contained false openings. BFS would then treat these openings as legitimate paths and generate routes that visually appeared to pass through walls.

Root Cause Analysis

The root cause of the issue was the assumption that a single pixel could accurately represent an entire 20×20 logical cell.

While this assumption held true in many cases, it became unreliable whenever small noisy regions existed inside wall cells. Since the classification decision depended entirely on one pixel, even a tiny artifact could change the state of an entire logical cell from wall to path.

This explained why BFS itself behaved correctly while the generated paths still appeared invalid when visualized on the original maze image.

Fix Applied

To eliminate this issue, I replaced the center-pixel classification method with a **majority-pixel approach**.

Instead of examining only one pixel, the entire 20×20 logical cell was analyzed.

For every cell:

- The number of red wall pixels was counted.
- If the count exceeded a threshold of 200 pixels, the cell was classified as a wall.
- Otherwise, the cell was classified as a walkable path.

This approach is significantly more robust because small noisy regions can no longer determine the classification of an entire logical cell.

By considering all pixels within a cell, the logical maze more accurately represents the true structure of the original image and prevents false openings from appearing in wall regions.

Final Validation

After updating the extraction logic, BFS was executed again using the corrected logical maze representation.

The resulting paths matched the actual maze structure and no longer crossed wall regions. Multiple mazes were tested to verify the correctness of the updated extraction method.

The corrected logical representation was then used in the final automated solver that processed all 100 maze images and generated the final submission outputs.

Code Reference: [Click Here \(solution_code.py\)](#)

Phase 4 – Solution Visualization

After correcting the maze extraction logic, the next step was to visualize the generated shortest path on the original maze image.

The BFS algorithm produces a sequence of logical grid coordinates representing the shortest route from the start cell to the goal cell. However, these coordinates exist only in the 40×40 logical maze representation and therefore must be converted back into image coordinates before visualization.

Each logical cell corresponds to a 20×20 pixel region in the original image. Using this mapping, every cell belonging to the shortest path was converted back into its corresponding image region and colored green.

Before applying the majority-pixel fix, visualizing the generated path revealed that some routes incorrectly crossed wall regions.

Code Reference: [Click Here \(draw_solution.py GitHub link\)](#)



Figure 12: Path generated using the center-pixel classification method. Due to incorrect cell classification near wall boundaries, the generated path appears to pass through wall regions.

After replacing the center-pixel approach with the majority-pixel classification method, the visualization matched the actual maze structure correctly.

The final implementation reconstructed the shortest path using the corrected logical maze representation and rendered the path on the original image.

Code Reference: [Click Here \(solution_code.py GitHub link\)](#)



Figure 13: Correct shortest path generated after applying majority-pixel classification. The path follows only valid maze corridors and accurately represents the true solution.

The visualization step served as an important validation mechanism throughout development. By comparing generated paths against the original maze image, it became possible to identify extraction errors, verify bug fixes and confirm the correctness of the final solution.

Final Solution Rendering

After validating that the corrected majority-pixel extraction method produced accurate paths, the final visualization pipeline was integrated into the complete solver.

The reconstructed shortest path was converted from logical grid coordinates back into image coordinates. Since each logical cell corresponds to a 20×20 pixel region in the original maze image, every cell belonging to the shortest path was mapped back to its corresponding image area.

To make the solution clearly visible, all cells belonging to the shortest path were colored green. This produced a solved maze image that visually highlights the route from the start cell to the goal cell.

The same visualization process was later applied automatically to all 100 maze images during batch processing.

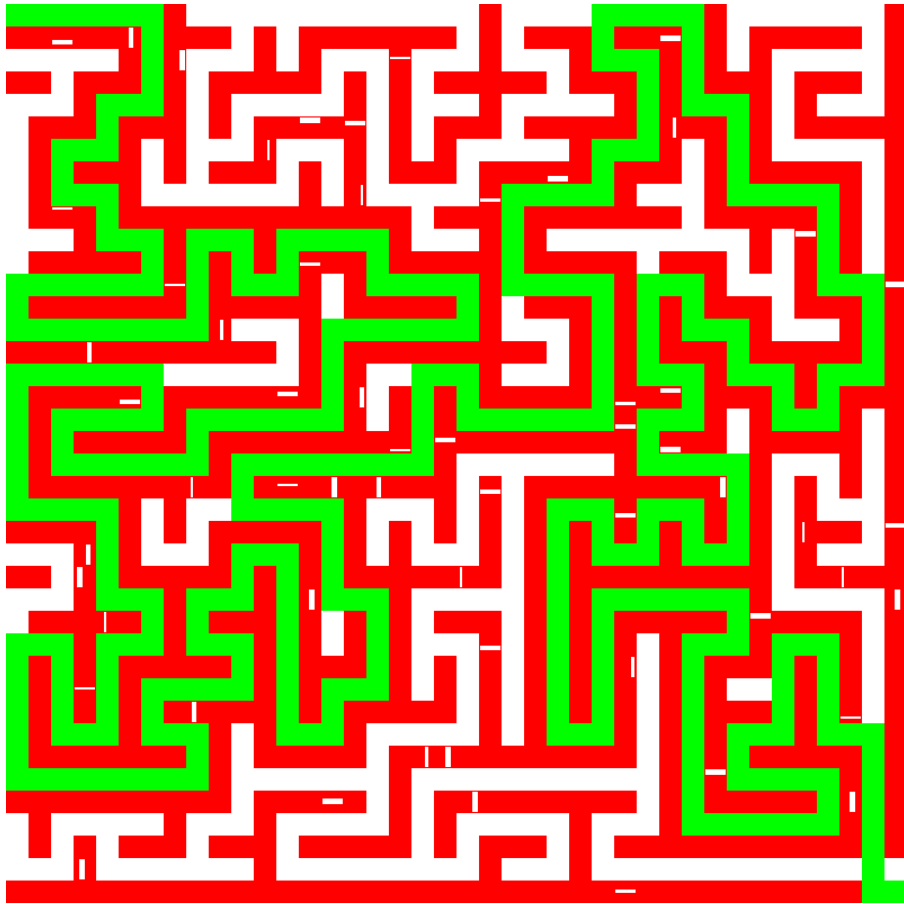


Figure 14: Final solved maze generated by the completed pipeline. The shortest path discovered by BFS is rendered in green and accurately follows valid maze corridors from the start cell to the goal cell.

The final visualization provided an intuitive way to verify correctness. By overlaying the computed path directly on the original maze image, it became easy to confirm that the solver respected wall boundaries and followed only valid paths.

This rendering step was also used to generate the output images stored inside the `answers` directory, which formed part of the final submission package.

Phase 5 – Full Automation, Impossible Maze Handling and Password Generation

After successfully developing and validating the maze-solving pipeline on individual maze images, the final step was to automate the entire process for the complete dataset.

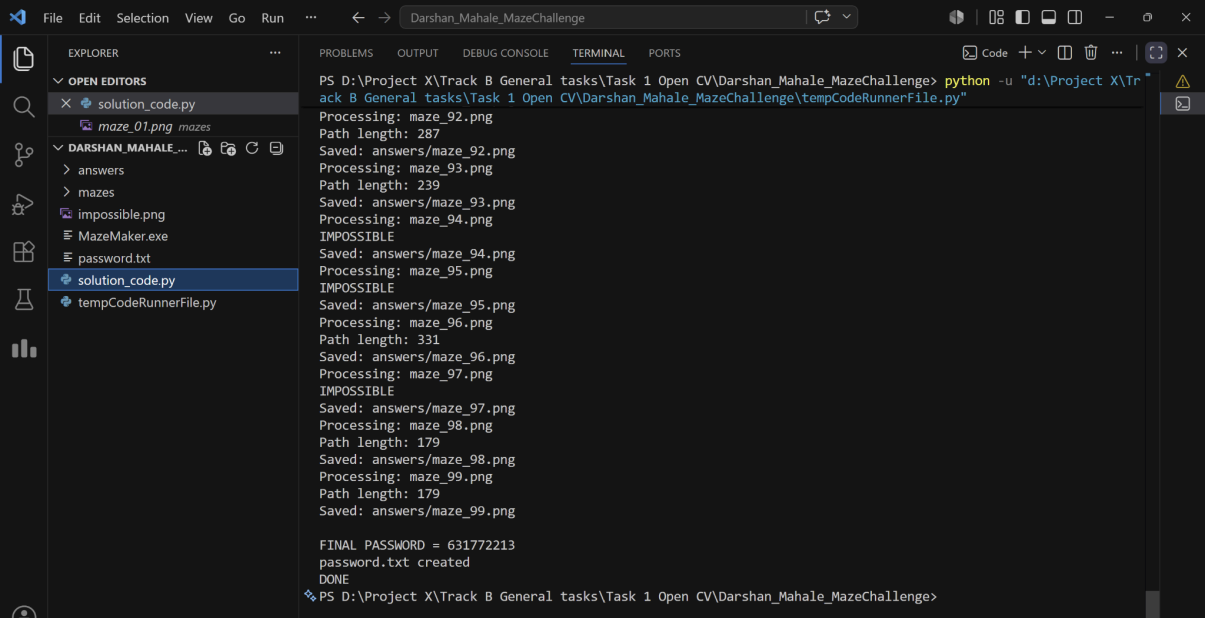
The challenge dataset contained 100 maze images. Manually solving each maze would have been impractical, so all previously developed components were integrated into a single automated pipeline.

For every maze image, the program performed the following sequence of operations:

- Load the maze image.
- Remove grayscale noise using the preprocessing logic developed earlier.
- Convert the image into a logical maze representation.
- Classify walls and paths using the majority-pixel approach.
- Execute BFS to find the shortest path.
- Generate the corresponding output image.
- Store the result inside the answers directory.

This allowed the entire dataset to be processed automatically without any manual intervention.

Code Reference: [Click Here \(solution_code.py GitHub link\)](#)



```
PS D:\Project X\Track B General tasks\Task 1 Open CV\Darshan_Mahale_MazeChallenge> python -u "d:\Project X\Tr
ack B General tasks\Task 1 Open CV\Darshan_Mahale_MazeChallenge\tempCodeRunnerFile.py"
Processing: maze_92.png
Path length: 287
Saved: answers/maze_92.png
Processing: maze_93.png
Path length: 239
Saved: answers/maze_93.png
Processing: maze_94.png
IMPOSSIBLE
Saved: answers/maze_94.png
Processing: maze_95.png
IMPOSSIBLE
Saved: answers/maze_95.png
Processing: maze_96.png
Path length: 331
Saved: answers/maze_96.png
Processing: maze_97.png
IMPOSSIBLE
Saved: answers/maze_97.png
Processing: maze_98.png
Path length: 179
Saved: answers/maze_98.png
Processing: maze_99.png
Path length: 179
Saved: answers/maze_99.png

FINAL PASSWORD = 631772213
password.txt created
DONE
PS D:\Project X\Track B General tasks\Task 1 Open CV\Darshan_Mahale_MazeChallenge>
```

Figure 15: Automated processing of maze images using the complete integrated pipeline.

Handling Impossible Mazes

While testing the automated pipeline, it became apparent that not every maze in the dataset contained a valid path from the start cell to the goal cell.

For such cases, BFS eventually exhausted all reachable cells without reaching the destination node. When this occurred, the maze was classified as impossible.

Instead of generating an incorrect solution image, a predefined image containing the message:

"IMPOSSIBLE, LACHURE SIR TRICKED ME"

was copied into the corresponding output file.

This ensured that every maze generated a valid output image while also satisfying the challenge requirements.

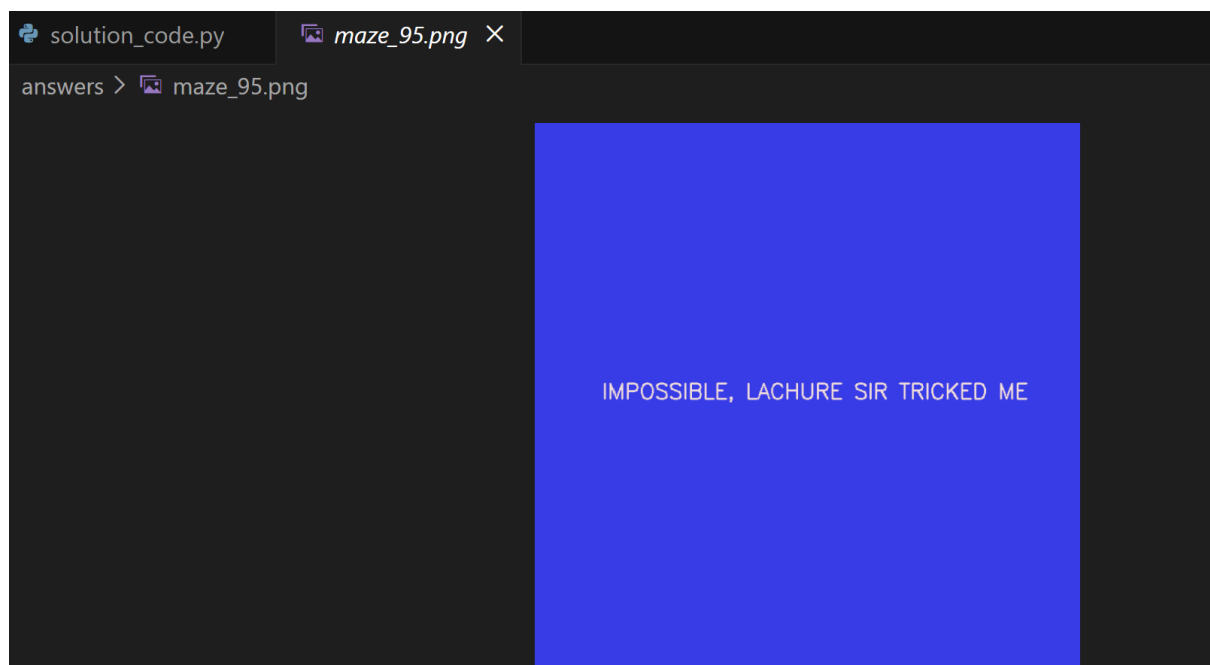


Figure 16: Output generated for a maze that does not contain a valid solution.

Password Generation

The final challenge requirement involved generating a password based on the shortest path lengths of all solvable mazes.

For every maze that contained a valid solution, the shortest path length produced by BFS was recorded. Impossible mazes were excluded from the calculation.

The path lengths of all solvable mazes were multiplied together to produce a single cumulative value. Since this number would become extremely large, the modulo operation specified in the challenge statement was applied repeatedly using:

1,000,000,007

This prevented integer overflow and produced the final password value required for submission.

The resulting password was automatically written to a text file named `password.txt`.

```
Path length: 179
Saved: answers/maze_99.png

FINAL PASSWORD = 631772213
password.txt created
DONE
PS D:\Project X\Track B General tasks\Task 1 Open CV\Darshan_I
```

Figure 17: Terminal output showing password generation and creation of password.txt.

Final Output Generation

At the end of execution, the program successfully processed all 100 maze images and generated:

- A complete answers directory containing solution images.
- Correct handling of impossible mazes.
- A password.txt file containing the final computed password.
- All files required for submission.

The final automated solution combined image preprocessing, maze extraction, pathfinding, visualization and password generation into a single workflow capable of solving the entire dataset efficiently.

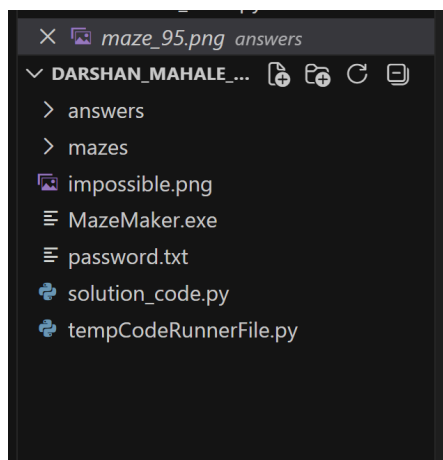


Figure 18: Final output directory containing generated answers and submission files.

Conclusion

This project successfully developed a complete automated pipeline for solving maze images using image processing and graph traversal techniques.

The solution began with an analysis of the dataset, where image properties and noise characteristics were investigated. A preprocessing stage was then designed to remove grayscale noise and convert the maze into a clean machine-readable representation. Through a series of experiments, the logical grid structure of the maze was identified and converted into a 40×40 matrix representation suitable for algorithmic processing.

Breadth First Search (BFS) was used to compute the shortest path between the start and goal cells. During development, a flaw in the initial center-pixel extraction approach was discovered through visual validation. Further investigation revealed that small white artifacts inside wall regions could cause incorrect cell classification. This issue was resolved by replacing the center-pixel approach with a majority-pixel classification method, resulting in a significantly more reliable maze representation.

After validating the correctness of the solution, the entire pipeline was automated to process all 100 maze images. The final implementation successfully handled both solvable and impossible mazes, generated visual solution outputs, and computed the required password according to the challenge specifications.

Overall, this project demonstrated the practical application of image preprocessing, data extraction, algorithm design, debugging, validation, and automation techniques to solve a real-world computer vision problem in a systematic and reliable manner.

Note on AI Usage

AI tools were used during this project for discussion, learning, brainstorming ideas, and improving the presentation of this report.

However, the overall approach, experimentation, debugging, testing, and final solution were developed through my own step-by-step analysis and implementation process. As shown throughout this report, the project involved multiple stages of investigation, validation, bug fixing, and refinement before reaching the final solution.

Repository and Source Code

The complete source code, intermediate development files, experimental scripts, and final implementation used in this project are available in the GitHub repository linked below.

[Click Here](#)

The repository contains all scripts developed during the project, including dataset analysis, image preprocessing, maze extraction, pathfinding, visualization, automation, and password generation components.